

ETF Analytics API – Requirements Document

(Version 1.0 – für Deployment auf Railway mit PostgreSQL)

1. Project Overview

1.1 Purpose

Die ETF Analytics API stellt strukturierte, normalisierte und analysierbare ETF-Daten über eine REST-API bereit.

Ziel ist es, Entwicklern, Fintechs und Research-Teams eine moderne, zuverlässige Datenquelle für:

- ETF-Stammdaten
- Holdings
- Sektor- und Länderallokationen
- Overlap-Analysen
- Portfolio-Exposure

bereitzustellen.

1.2 Scope (MVP)

- Datenimport (ETL) für ausgewählte ETF-Provider (Vanguard, iShares, UBS). Für den MVP können Zufallsdaten genutzt werden.
 - Speicherung in PostgreSQL.
 - REST-API auf FastAPI-Basis.
 - Deployment auf Railway (API-Service + Worker + Postgres).
 - Basis-Analytics: Overlap, Exposure, Similarity.
-

2. Target Users

2.1 Primary Users

- Fintech-Entwickler
- Robo-Advisor
- Portfolio-Analyse-Tools
- Vermögensverwalter

- Finance-Educators

2.2 Key Use Cases

- Automatisiertes Abrufen von ETF-Daten
 - Portfolio-Analyse
 - Risiko- und Exposure-Analysen
 - Vergleich ähnlicher ETFs
 - Backtesting-Vorbereitung
-

3. Functional Requirements

3.1 Data Model

3.1.1 ETF

- `id` (UUID)
- `isin`
- `ticker`
- `name`
- `provider`
- `domicile`
- `replication_method`
- `ter`
- `fund_size`
- `benchmark`
- `currency`
- `listings` (JSON)

3.1.2 Holdings

- `id`
- `etf_id`
- `date`
- `instrument_isin`
- `instrument_name`
- `weight`
- `country`
- `sector`

3.1.3 Allocations

- `id`
- `etf_id`
- `date`
- `type` (sector | country | currency)
- `bucket`
- `weight`

3.1.4 Performance (optional MVP)

- `id`
 - `etf_id`
 - `date`
 - `nav`
 - `close_price`
 - `currency`
 - `dividend`
-

3.2 ETL Requirements

3.2.1 Data Sources

- Provider-Webseiten (HTML/CSV)
- Factsheets (PDF-Parsing optional)
- Holdings-Downloads (CSV bevorzugt)

3.2.2 ETL Pipeline

- Scraper → Parser → Normalizer → Validator → DB Writer
- Jobs laufen täglich oder wöchentlich
- Idempotente Updates
- Historische Daten werden versioniert gespeichert

3.2.3 Validation Rules

- Holdings sum $\approx$ 100 %
 - Mandatory fields present
 - ISIN format check
 - Duplicate detection
-

3.3 API Requirements

3.3.1 Authentication

- API-Key (Header: `X-API-Key`)
- Rate-Limiting pro Key

3.3.2 Public Endpoints

ETF

- `GET /etfs`
- `GET /etfs/{id}`

Holdings

- `GET /etfs/{id}/holdings?date=YYYY-MM-DD`

Allocations

- `GET /etfs/{id}/allocations?type=sector|country&date=...`

Performance

- `GET /etfs/{id}/performance?from=&to=`

3.3.3 Analytics Endpoints

Overlap

- `POST /analytics/overlap`
Input: Liste von ETFs
Output: Overlap-Matrix + gemeinsame Holdings

Pairwise Overlap

- `GET /analytics/overlap/{etfA}/{etfB}`

Portfolio Exposure

- `POST /analytics/exposure`
Input: `{etf_id, weight}`
Output: aggregierte Sektor-/Länderallokation

Similarity

- `GET /analytics/similar/{etf_id}`
-

6. Deployment Requirements (Railway)

6.1 Services

- API Service (Docker)
- Worker Service (Docker)
- PostgreSQL
- Optional Redis

6.2 Environment Variables

- DATABASE_URL
 - API_KEY_SECRET
 - ETL_SCHEDULE
 - LOG_LEVEL
-

7. MVP Deliverables

- Datenbank-Schema
 - ETL für 10–20 ETFs
 - API-Endpoints für Stammdaten, Holdings, Allocations
 - Overlap-Engine
 - Deployment auf Railway
 - API-Key-System
-

8. Future Extensions

- Tracking Difference Analytics
- Faktor-Exposure
- Backtesting-Module
- Web-Dashboard
- Webhooks für Datenänderungen

9. Web-Dashboard

- ETF-Übersicht
- ETF-Detailansicht
- Holdings-Übersicht
- Allokation-Charts
- API-Konfiguration